Where Does the Gain in Dynamic Convolution Come From?

A controlled study of routing, capacity and width on CIFAR-100 and Tiny-ImageNet

Ankit Kumar

Mechatronics Department, University of Agder, Grimstad, Norway

24 September 2026; revised 30 September 2026

Abstract

Dynamic convolution replaces one kernel with a bank of K kernels mixed by input-dependent weights. Its gains are usually credited to input-adaptive kernel selection, but published comparisons set it against a network with K times fewer parameters, so routing and capacity are confounded. We separate them with a constant-mixture control, a mismatched-input control and a frozen random router, across 132 training runs on CIFAR-100 and Tiny-ImageNet. Four results hold throughout. The kernel bank on its own contributes nothing: in all five conditions tested, a network whose K kernels are mixed uniformly is statistically equivalent to a plain network at a ± 0.5 pp margin. Where dynamic convolution helps, the routing term accounts for the effect. At matched parameter budgets, a plain wider network beats the routed one at all three budgets we could match, by 2.3 to 4.3 pp, at a fraction of the training cost. And routing entropy, the statistic used to show that a router is active, does not track the benefit: a router 14 times flatter by entropy delivered a larger gain than a sharper one. The size of the routing effect is less stable. On CIFAR-100 it vanishes at the standard width of 64, under both a constant and an annealed softmax temperature. On Tiny-ImageNet it does not (+2.282 pp at width 64). Before running that experiment we recorded a prediction that the effect would vanish there because the network memorises its training data. The prediction failed, and we report it as failed.

1 Introduction

A dynamic convolution layer stores K kernels and combines them per input. For an input feature map F and kernel bank $\{\mathcal{W}_k\}_{k=1}^{K}$, the layer computes

$$\mathcal{W}_{\text{dyn}}(F) = \sum_{k=1}^{K} z_k(F)\,\mathcal{W}_k, \qquad \mathbf{z} = \operatorname{softmax}\!\left(\frac{g(\phi(F))}{\tau}\right), \tag{1}$$

where ϕ summarises the feature map, g is a small multilayer perceptron and τ is a softmax temperature. Nearly every published variant sets ϕ to global average pooling (Yang et al., 2019; Chen et al., 2020).

The usual account of why this helps is that $\mathbf{z}$ adapts the kernel to the input. A second account is available and is rarely tested. The bank multiplies the stored convolution parameters by K, and published comparisons are made against a network with K times fewer parameters. Under that comparison, a gain from routing and a gain from extra capacity look identical.

This report separates the two. We compare the standard routed layer against a constant mixture that has the same bank and no routing, a router fed a different image's descriptor, a router frozen at random initialisation, and a plain network. The comparison spans three widths, three bank sizes, two temperature schedules and two datasets, for 132 training runs and 204 GPU-hours on one RTX 4070 Ti. Every comparison is paired by seed and tested for equivalence as well as for difference, so that a null result is a claim with a stated margin rather than a failure to reject.

Four findings hold across everything we ran:

1. The kernel bank contributes nothing on its own, in all five conditions tested across both datasets (Section 7).
2. Where dynamic convolution helps, the routing term accounts for the effect.
3. At matched parameter budgets, spending the parameters on width beats spending them on a kernel bank, at all three budgets we could match (Section 10).
4. Routing entropy does not measure whether routing is useful (Section 12).

The size of the routing effect is less stable than any of these. On CIFAR-100 it peaks at width 32 and vanishes at width 64. We attributed that null to the network memorising its training set, and before extending the study to Tiny-ImageNet at width 64 we wrote down a prediction that the effect would vanish there as well. It did not. We report that prediction as failed (Section 9) and treat the CIFAR-100 width-64 null as an observation we cannot yet explain.

2 Related work

Dynamic convolution. CondConv (Yang et al., 2019) and DY-Conv (Chen et al., 2020) introduced input-dependent mixtures of kernels. ODConv (Li et al., 2022) extends the attention across several kernel dimensions, and KernelWarehouse (Li and Yao, 2024) reorganises the bank so that far larger K becomes affordable. The reported gains are large: for a ResNet-18 on ImageNet at four times the kernel parameters, Li and Yao (2024) report top-1 improvements of 3.38 pp for DY-Conv, 4.01 pp for ODConv and 5.61 pp for KernelWarehouse under their training recipe. FDConv (Chen et al., 2025) observes that the parallel kernels of DY-Conv tend toward highly similar frequency responses, so that the extra parameters buy little adaptability. Our finding that the bank contributes nothing on its own is consistent with that observation. Spatially resolved variants such as DRConv (Chen et al., 2021) route on local rather than global signals and fall outside the controls used here.

Optimisation difficulty. Li et al. (2021) identify the joint optimisation of dynamic attention and static kernels as the central difficulty of dynamic convolution and propose a matrix decomposition that eases it. DY-Conv addresses the same difficulty by annealing the softmax temperature from 30 to 1 early in training (Chen et al., 2020). Section 11 tests that schedule.

Re-parameterisation. A constant mixture of K kernels is a linear re-parameterisation of a single kernel. Such re-parameterisations can change optimisation without changing the function class (Arora et al., 2018), and ACNet and RepVGG exploit this by training a richer block and merging it into one kernel at inference (Ding et al., 2019, 2021). Our `uniform` arm is exactly such a mergeable block.

Memorisation. Networks of the size used here can fit their training sets completely, even under random labels (Zhang et al., 2017). That motivated the saturation hypothesis tested in Section 9.

Equivalence testing. The two one-sided tests procedure (Schuirmann, 1987) asserts that a difference lies within a stated margin; Lakens (2017) give a practical treatment. It is standard in bioequivalence studies and rare in this literature, where a large p-value from a difference test is often read as evidence of no effect.

3 Method

3.1 Routing signals

All arms share Eq. (1), the backbone, the optimiser and the seeds, and differ only in how $\mathbf{z}$ is produced. Write $\mu(F) \in \mathbb{R}^C$ for the channel-wise global average.

`static` No dynamic layer: a plain ResNet-18, $K = 1$.

`uniform` $g \equiv \mathbf{0}$, so $\mathbf{z} = (1/K, \ldots, 1/K)$ for every input. The bank exists and is trained; nothing is routed.

`gap` $\mathbf{z} = \mathrm{softmax}(g(\mu(F))/\tau)$ with g trained. The standard layer.

`shuffled` As `gap`, but sample i is routed with the descriptor of sample $\pi(i)$ for a permutation π redrawn on every forward pass, at evaluation as well as training. The distribution of $\mathbf{z}$ is preserved and the pairing between image and kernel is destroyed.

`frozen_random` As `gap`, with every weight of g frozen at its random initialisation. Routing varies with the input but is never optimised.

3.2 The constant mixture is a single kernel

Proposition 1. *If $\mathbf{z} = (1/K, \ldots, 1/K)$ for every input, the layer computes $F * \bar{\mathcal{W}}$ with $\bar{\mathcal{W}} = \frac{1}{K}\sum_k \mathcal{W}_k$, and its effective parameter count equals that of one static kernel.*

Proof. The mixture in Eq. (1) is constant, so $\mathcal{W}_{\text{dyn}} = \bar{\mathcal{W}}$ for every F. □

At inference `uniform` is therefore a plain ResNet-18. At width 64 it stores 44.18M parameters and has an effective count of 11.22M. It spans the same function class as `static`, so any difference between the two is an effect of optimisation. That makes it the right control for the extra parameters of the bank.

3.3 Two exact implementations

Convolution is linear in its weights, so

$$F * \Big(\sum_k z_k \mathcal{W}_k \Big) = \sum_k z_k \, (F * \mathcal{W}_k). \tag{2}$$

The left side builds a per-sample kernel and runs one grouped convolution; at width 64 and batch 128 its aggregated weights alone take about 1.2 GB per stage-4 layer. The right side runs K ordinary convolutions and mixes the outputs, which costs K times the arithmetic but no per-sample weights. Every run uses the right side. The test suite asserts that the two agree to 10^{-10} in double precision across four routing signals.

3.4 Training protocol

Data. CIFAR-100 (Krizhevsky, 2009), 32×32, 100 classes, at widths 16, 32 and 64. Tiny-ImageNet (Le and Yang, 2015), 64×64, 200 classes, at widths 32 and 64. The stem is the same on both: a 3×3 stride-1 convolution with no max-pool. At 64×64 this leaves an 8×8 final feature map rather than 4×4 and roughly quadruples the cost of the deeper stages, but changing the stem would change the network between datasets. Crop padding is 4 pixels at 32 and 8 at 64.

Network and recipe. ResNet-18 (He et al., 2016) in which all 16 3×3 convolutions inside residual blocks are dynamic; the stem and the 1×1 shortcuts stay static. $K = 4$ unless stated. SGD with Nesterov momentum 0.9, learning rate 0.1, weight decay 5×10^{-4}, batch 128, five warm-up epochs then cosine decay over 150 epochs. Augmentation is random cropping and horizontal flipping only. Mixed precision in bfloat16. Six seeds, 0 to 5, shared by every arm. Batch 128 held at Tiny-ImageNet width 64 after a one-epoch memory probe, so the two Tiny-ImageNet widths are directly comparable.

Temperature. $\tau = 1$ throughout, except in Section 11, where τ falls linearly from 30 to 1 over the first ten epochs and stays at 1 thereafter.

4 Statistical protocol

Equivalence. Several conclusions are null conclusions, and a large p from a difference test does not establish one. For paired differences d_i with mean $\bar{d}$ and standard error $s_{\bar{d}}$ we use two one-sided tests against a margin δ,

$$p_{\text{TOST}} = \max \Big\{ 1 - T_{n-1}\Big(\tfrac{\bar{d}+\delta}{s_{\bar{d}}}\Big),\; T_{n-1}\Big(\tfrac{\bar{d}-\delta}{s_{\bar{d}}}\Big) \Big\}, \tag{3}$$

and read $p_{\text{TOST}} < 0.05$ as a positive claim that the true difference lies within $\pm\delta$. We fixed $\delta = 0.5$ pp before the first run. The pre-registration justified this by reference to reported gains at CIFAR scale without citing a source. The nearest documented figures we have since found are the ImageNet ResNet-18 gains of 3.38 to 5.61 pp quoted above, and the margin sits an order of magnitude below them.

Why six seeds. The two-sided Wilcoxon signed-rank test (Wilcoxon, 1945) on n pairs admits only 2^n equally likely sign assignments, so its smallest attainable p is 2^{1-n}: 0.25 at $n = 3$, 0.0625 at $n = 5$, 0.031 at $n = 6$. Six is the smallest count at which it can reach 0.05. We report it beside the paired t-test throughout.

Power. For `gap − uniform` on CIFAR-100 at width 64 the paired standard deviation is 0.257 pp, giving a minimum detectable difference of 0.270 pp at $n = 6$. Simulated power (2×10^5 draws) is 48.5 % for a true effect of 0.25 pp, 96.3 % at 0.5 pp and 100.0 % at 1.0 pp. The width-64 null on CIFAR-100 is not a power artefact.

Pre-registration. Arms, recipe, primary metric, seeds, margin and the decision table were written down before the first run finished. The primary metric is final-epoch top-1 accuracy, since best-epoch accuracy maximises over a noisy sequence. The Tiny-ImageNet width-64 prediction of Section 9 was recorded in writing, dated, before any of those runs. An expectation about Tiny-ImageNet width 32 was stated before those runs but not recorded; we give it correspondingly less weight.

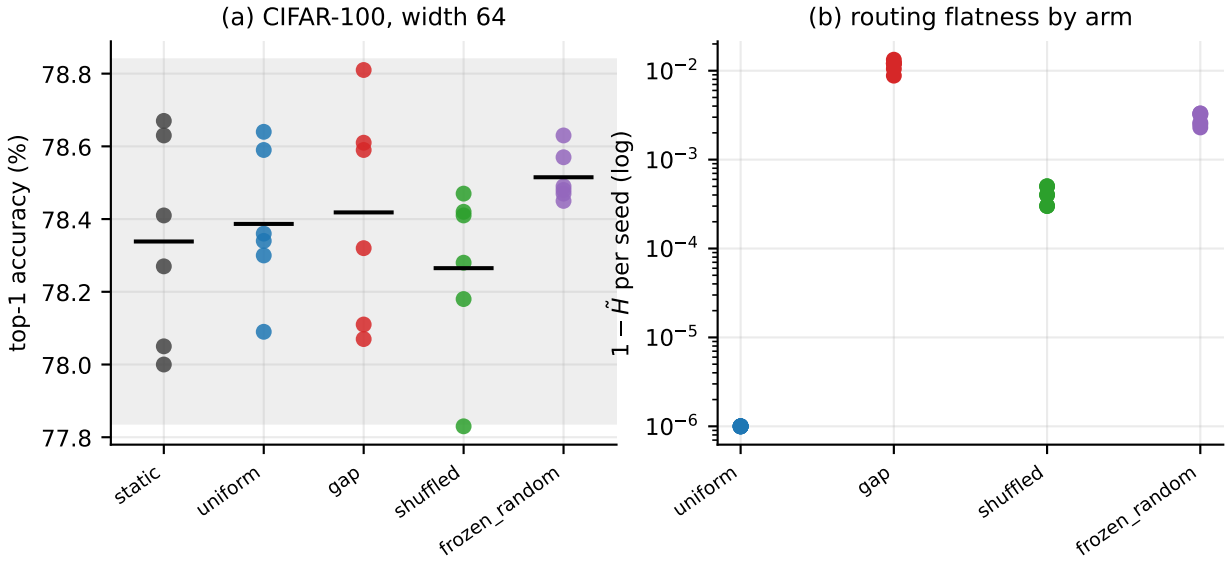


Figure 1: CIFAR-100, width 64. (a) Every run of the five arms; bars are arm means and the shaded band is ±0.5 pp around `static`. (b) Router flatness $1 - \tilde{H}$ per seed, log scale; `uniform` is exactly zero and is drawn at the axis floor.

Table 1: Bank and routing terms, paired by seed. The bank term is equivalent to zero in all five conditions.

		bank (`uniform−static`)			routing (`gap−uniform`)		total	
dataset	width	Δ (pp)	p_{TOST}	verdict	Δ (pp)	p	Δ (pp)	p
CIFAR-100	16	-0.223	0.0251	equivalent	+0.755	0.0023	+0.532	0.0023
CIFAR-100	32	+0.175	0.0242	equivalent	+0.978	0.0010	+1.153	$< 10^{-4}$
CIFAR-100	64	+0.048	0.0142	equivalent	+0.032	0.7752	+0.080	0.6158
Tiny-ImageNet	32	+0.107	0.0218	equivalent	+2.078	$< 10^{-4}$	+2.185	$< 10^{-4}$
Tiny-ImageNet	64	+0.053	0.0157	equivalent	+2.228	$< 10^{-4}$	+2.282	$< 10^{-4}$

5 CIFAR-100 at the standard width

At width 64 every arm is equivalent to a plain ResNet-18 (Table 10, Figure 1). The largest gap between arm means is 0.250 pp, against a median within-arm seed standard deviation of 0.238 pp. A one-way ANOVA across the five arms gives $F(4, 25) = 0.954$, $p = 0.450$.

A router whose weights were never trained matches a trained one (`frozen_random` − `gap` = +0.097 pp, $p = 0.443$, $p_{\text{TOST}} = 0.0089$), and so does a router that routes each image with another image's descriptor. The trained router is not degenerate: its per-seed mean entropy stays below that of the controls (Figure 1b). It discriminates between kernels, and at this width the discrimination changes nothing.

6 The capacity sweep on CIFAR-100

Reducing width changes the answer (Figure 2). `gap` − `static` is +0.532 pp at width 16 ($p = 0.0023$), +1.153 pp at width 32 ($p < 10^{-4}$) and +0.080 pp at width 64 ($p = 0.6158$). The effect is not monotone in capacity: it peaks at width 32. `uniform` is equivalent to `static` at all three widths. Figure 10 in the appendix shows every run.

7 The kernel bank contributes nothing

Splitting the total effect `gap` − `static` into a bank term and a routing term separates the two accounts of the introduction. Table 1 and Figure 3 give the split in all five conditions that have a `uniform` arm.

The bank term is equivalent to zero in every one of them, on both datasets, with p_{TOST} between 0.014 and 0.025. The largest bank term observed is +0.175 pp. Since the bank contributes nothing within the margin, the total effect is the routing effect: on Tiny-ImageNet at width 64, for example, routing is +2.228 pp of a +2.282 pp total. At CIFAR-100 width 16 the bank term is slightly negative (-0.223 pp, equivalent) and the routing term therefore exceeds the total, which is why we report no percentage share.

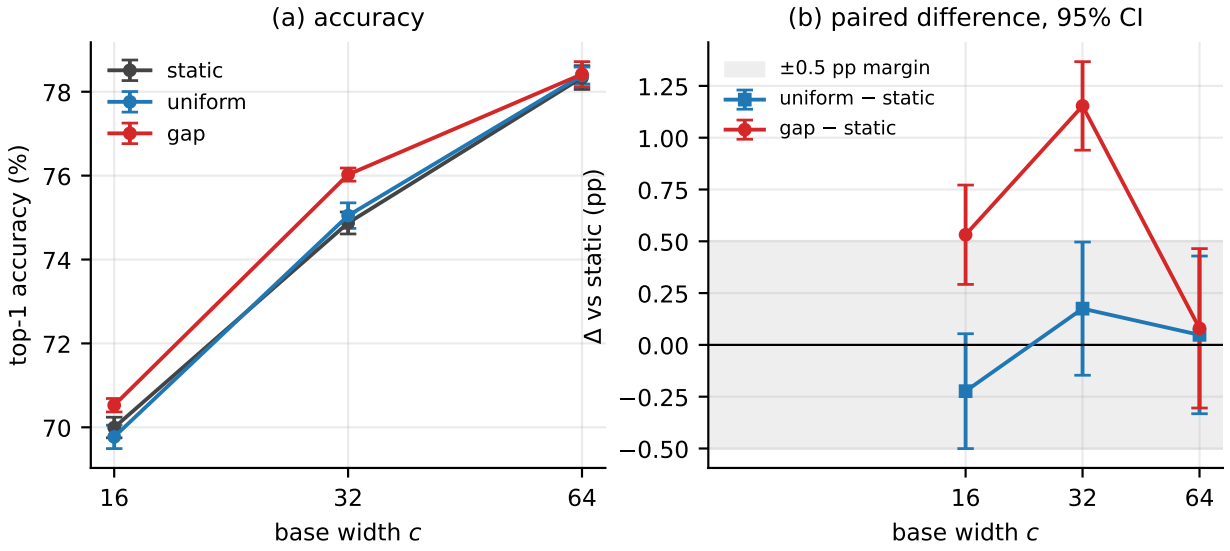


Figure 2: CIFAR-100. (a) Accuracy against base width, ±1 standard deviation over six seeds. (b) Paired difference from `static` with 95 % confidence intervals.

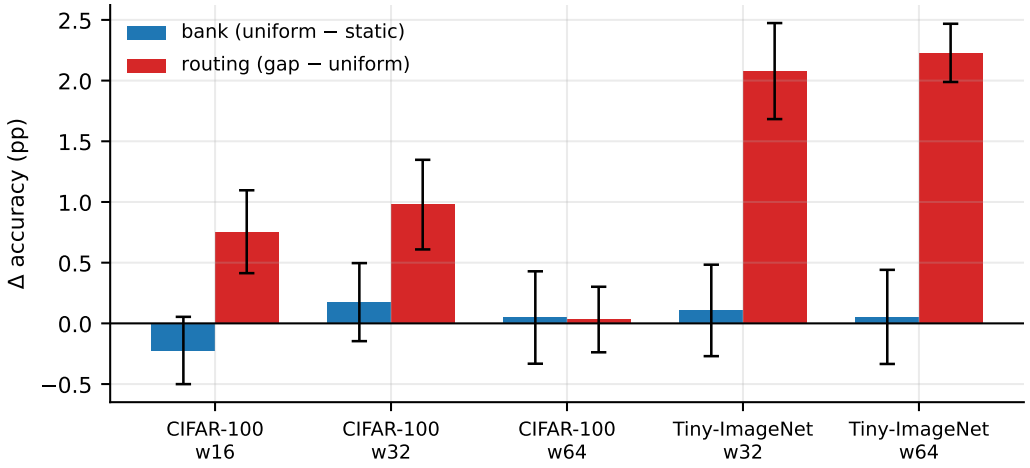


Figure 3: Decomposition of the effect of dynamic convolution into the bank (`uniform` − `static`) and routing (`gap` − `uniform`), five conditions, 95 % CIs.

8 Bank size

The width-32 peak was measured at $K = 4$; Table 2 repeats it at $K = 2$ and $K = 8$. All three bank sizes beat `static` by a similar margin, and no pair of them is distinguishable (Table 3): $K = 8$ over $K = 2$ is +0.098 pp ($p = 0.409$). The benefit is already complete at $K = 2$. Going from $K = 2$ to $K = 8$ multiplies stored parameters by 3.9 and training time by 2.5 (Figure 4).

9 Tiny-ImageNet, and a failed prediction

9.1 Width 32

On Tiny-ImageNet at width 32, `gap` − `static` is +2.185 pp (95 % CI [+1.781, +2.589], $p < 10^{-4}$), against +1.153 pp on CIFAR-100 at the same width. The bank term is +0.107 pp and equivalent to zero. Before these runs we expected the effect to be larger than on CIFAR-100, since the static arm trains to a higher loss (0.1194 against 0.0170) and so sits further from memorisation. That expectation held, but it was stated and not recorded, it predicted only a direction, and we do not treat it as strong evidence.

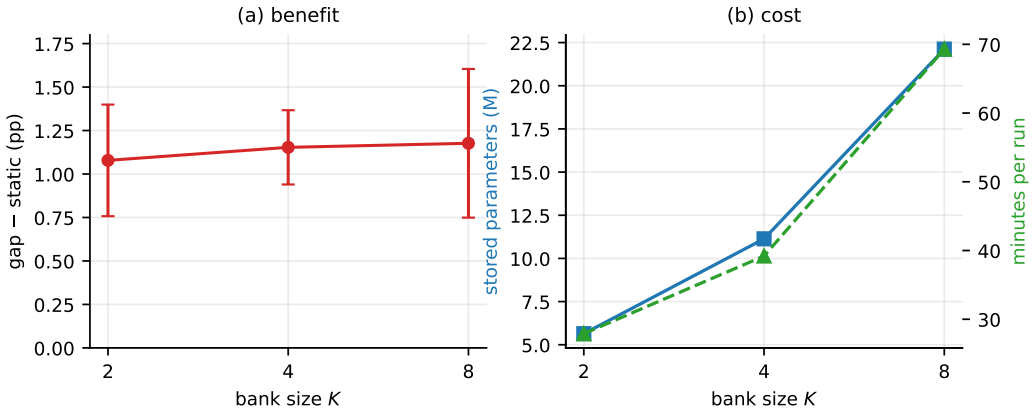


Figure 4: CIFAR-100, width 32. (a) Effect over `static` for $K \in \{2, 4, 8\}$, 95 % CIs. (b) Stored parameters and training time.

Table 2: Bank size at CIFAR-100 width 32. Differences are against the `static` row.

K	accuracy	sd	Δ (pp)	95 % CI	p	params (M)	min/run
2	75.950	0.148	+1.078	[+0.758, +1.399]	0.0003	5.64	27.9
4	76.025	0.156	+1.153	[+0.940, +1.367]	$< 10^{-4}$	11.13	39.2
8	76.048	0.242	+1.177	[+0.749, +1.604]	0.0009	22.12	69.3
`static`	74.872	0.262	–	–	–	2.82	8.6

9.2 Width 64: the recorded prediction

The saturation hypothesis says the routing effect vanishes once a network memorises its training data. On CIFAR-100 the width-64 null coincides with a final training loss of 0.0080. Before running Tiny-ImageNet at width 64 we recorded, in writing and dated, the prediction in Table 4: that the effect would fall below 0.5 pp and be equivalent to zero, and that the static arm would train to a loss below 0.05.

The loss condition held (0.0210) and the effect did not vanish: `gap − static` $= +2.282$ pp, 95 % CI [+1.952, +2.611], $p = 1.0 \times 10^{-5}$. The prediction failed. The bank term is again equivalent to zero (+0.053 pp, $p_{\text{TOST}} = 0.0157$), so the effect is carried by routing, and it is as large at width 64 as at width 32.

9.3 What remains of the saturation account

Table 5 orders the five conditions by the final training loss of the static arm. The effects do not follow that order (Figure 6b). Tiny-ImageNet at width 64 trains to a loss close to CIFAR-100 at width 32 and shows twice the effect; CIFAR-100 at width 16 trains to the highest loss and shows one of the smallest. A threshold near 0.01 would separate the one null from the four positive results, but it would be chosen after seeing the data, and we do not adopt it.

What survives is narrower. On CIFAR-100 the routing effect vanishes at width 64, and that result is well powered and holds under two temperature schedules. It does not replicate on Tiny-ImageNet, and training loss does not explain it. Plausible differences between the datasets include the input resolution, the final feature-map size and the number of classes; none of them was isolated here.

10 Matched parameter budgets

Because widths differ by a factor of two and the bank multiplies convolution parameters by four, a routed network at one width stores almost exactly as many parameters as a plain network at the next width up. That gives three matched comparisons without any extra runs (Table 6, Figure 7).

The plain network wins all three: by +4.345 pp at 2.8M parameters and +2.313 pp at 11.2M on CIFAR-100, and by +2.520 pp at 11.2M on Tiny-ImageNet. It also trains faster in each case; on Tiny-ImageNet, 129.0 min per run against 284.2 min. A fourth, unmatched comparison points the same way: the $K = 8$ network at width 32 stores 22.1M parameters, twice the plain width-64 network, and trails it by 2.29 pp.

Table 3: Pairwise comparisons between bank sizes.

comparison	Δ (pp)	95 % CI	p	p_{TOST}	verdict
$K = 4$ vs $K = 2$	+0.075	[-0.087, +0.237]	0.2878	0.0005	equivalent
$K = 8$ vs $K = 4$	+0.023	[-0.286, +0.332]	0.8538	0.0053	equivalent
$K = 8$ vs $K = 2$	+0.098	[-0.182, +0.379]	0.4091	0.0072	equivalent

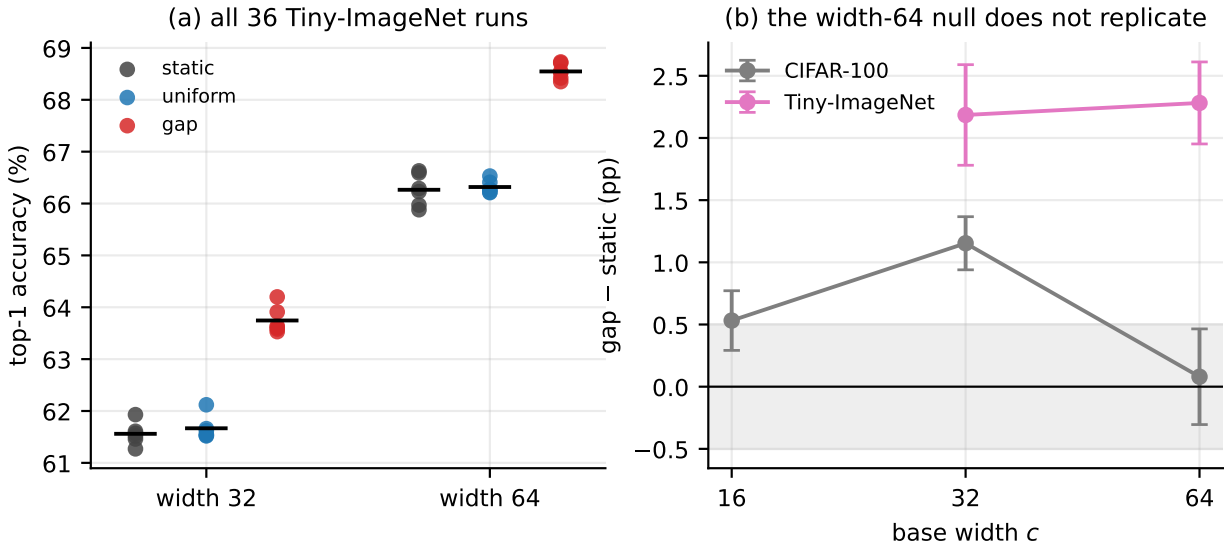


Figure 5: (a) All 36 Tiny-ImageNet runs. (b) `gap` − `static` by width on both datasets, 95 % CIs. The CIFAR-100 width-64 null does not appear on Tiny-ImageNet.

11 Temperature schedule

With $\tau = 1$ the softmax can be close to one-hot from the first step, starving most of the bank of gradient. DY-Conv anneals τ from 30 to 1 for this reason (Chen et al., 2020). If the CIFAR-100 width-64 null came from an undertrained bank, it would say nothing about dynamic convolution, so we repeated `gap` at all three widths with the annealed schedule (Table 7).

The width-64 null survives: annealed `gap` against `static` is +0.032 pp with $p_{\text{TOST}} = 0.0025$, a tighter equivalence than the 0.0188 obtained at $\tau = 1$. Annealing is otherwise neutral at width 64 (-0.048 pp, $p = 0.7449$) and width 16 (+0.115 pp, $p = 0.4313$), and harmful at width 32 (-0.682 pp, $p = 0.0017$), where it also leaves a higher final training loss (0.0173 against 0.0132). Backbone, dataset, bank size and schedule length all differ from the DY-Conv setting, so this is not evidence against their result; it is evidence that the CIFAR-100 null does not depend on the choice of schedule.

At width 64 the annealed router went flat during the high-τ phase and stayed flat for the remaining 140 epochs: it is 63 times flatter than the constant router ($1 - \tilde{H} = 0.00018$) and equivalent to `uniform` (-0.017 pp, $p_{\text{TOST}} = 0.0007$). This follows from Proposition 1. A flat mixture trains the bank toward its own mean, and a homogeneous bank leaves the router nothing to discriminate between. At width 16 the same schedule did not produce that outcome, which the next section takes up.

12 Routing entropy does not measure routing usefulness

Normalised routing entropy $\tilde{H} = H(\mathbf{z})/\log K$ is the statistic this literature reports to show that a router is doing something. Table 8 sets it against the benefit each router delivers over `uniform` in six conditions.

At width 16 the two schedules give a counterexample (Figure 9). The constant router has $1 - \tilde{H} = 0.03983$ and beats `uniform` by +0.755 pp. The annealed router has $1 - \tilde{H} = 0.00282$, 14 times flatter, and beats `uniform` by +0.870 pp ($p = 0.0044$). A router that looks nearly inactive on the standard diagnostic delivered more than one that looks clearly active.

The reason is that entropy compresses the signal quadratically. Writing $z_k = 1/K + \varepsilon_k$ with $\sum_k \varepsilon_k = 0$ and expanding to second order,

$$1 - \tilde{H} \approx \frac{K}{2 \log K} \sum_k \varepsilon_k^2, \tag{4}$$

Table 4: The prediction recorded before the Tiny-ImageNet width-64 runs, and the outcome. Both conditions were stated as required.

	prediction	measured	outcome
(a)	`gap − static` below 0.5 pp, equivalent by TOST	+2.282 pp, $p = 1.0 \times 10^{-5}$, $p_{\mathrm{TOST}} =$ 1.0000	failed
(b)	static final training loss below 0.05	0.021	held

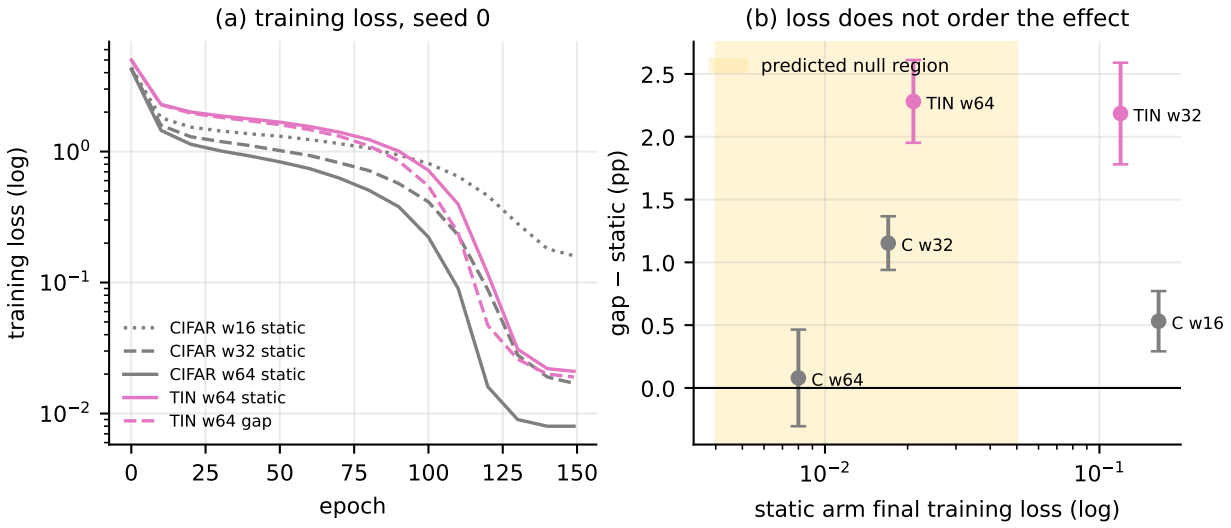


Figure 6: (a) Training loss of static and gap arms, seed 0, log scale. (b) Routing effect against the final training loss of the static arm in all five conditions. The shaded band is the region the prediction associated with a null.

so the deviations that carry input-dependent information enter squared. At $1 - \tilde{H} = 0.00282$ the root-mean-square deviation from $1/K$ is about 0.022, so typical mixing weights lie in [0.228, 0.272], roughly 9 % of $1/K$. Entropy reports that as 0.997.

Entropy is not meaningless. The annealed router at width 64, at $1 - \tilde{H} = 0.00018$, has collapsed in the sense that matters: it is equivalent to `uniform`. But across the six conditions of Table 8, sorting by $1 - \tilde{H}$ does not sort the benefit, and the two schedules do not lie on one curve (Figure 8c). A value of $\tilde{H}$ below one is not evidence that routing is useful, and a value near one is not evidence that it is not.

13 Threats to validity

Scale. Both datasets are low-resolution and every run fits on one consumer GPU. Behaviour at ImageNet scale is untested and cannot be tested on this hardware.

One backbone. ResNet-18 throughout. A replication of the full 132-run program on MobileNetV2, VGG-16, ResNet-50 and ConvNeXt-Tiny, with the same recipe, arms, seeds and dynamic-layer rule, was pre-registered on 30 September 2026 and is reported separately.

Initialisation. Static convolutions in the `static` arm, and the static stem, shortcut and head layers in every arm, keep PyTorch's default initialisation (Kaiming-uniform with $a = \sqrt{5}$), while every kernel of every dynamic bank is initialised Kaiming-normal with fan-out. The `uniform−static` bank term therefore compares two initialisations as well as a bank against a single kernel. The routing term, `gap−uniform`, is not affected: both arms have identically initialised banks. The replication on four more backbones initialises static and bank kernels with one scheme.

Not a reproduction of a published system. The layer is a dynamic convolution of the CondConv and DY-Conv form. No published implementation of ODConv, FDConv or KernelWarehouse was run, and the conclusions concern dynamic convolution of this form.

Global routing. Every router here uses a global descriptor. Spatially resolved routing is not covered.

Table 5: Five conditions ordered by the final training loss of the static arm.

dataset	width	static train loss	`gap` − `static` (pp)	95 % CI	verdict
CIFAR-100	64	0.0080	+0.080	[-0.305, +0.465]	equivalent
CIFAR-100	32	0.0170	+1.153	[+0.940, +1.367]	higher
Tiny-ImageNet	64	0.0210	+2.282	[+1.952, +2.611]	higher
Tiny-ImageNet	32	0.1194	+2.185	[+1.781, +2.589]	higher
CIFAR-100	16	0.1643	+0.532	[+0.292, +0.771]	higher

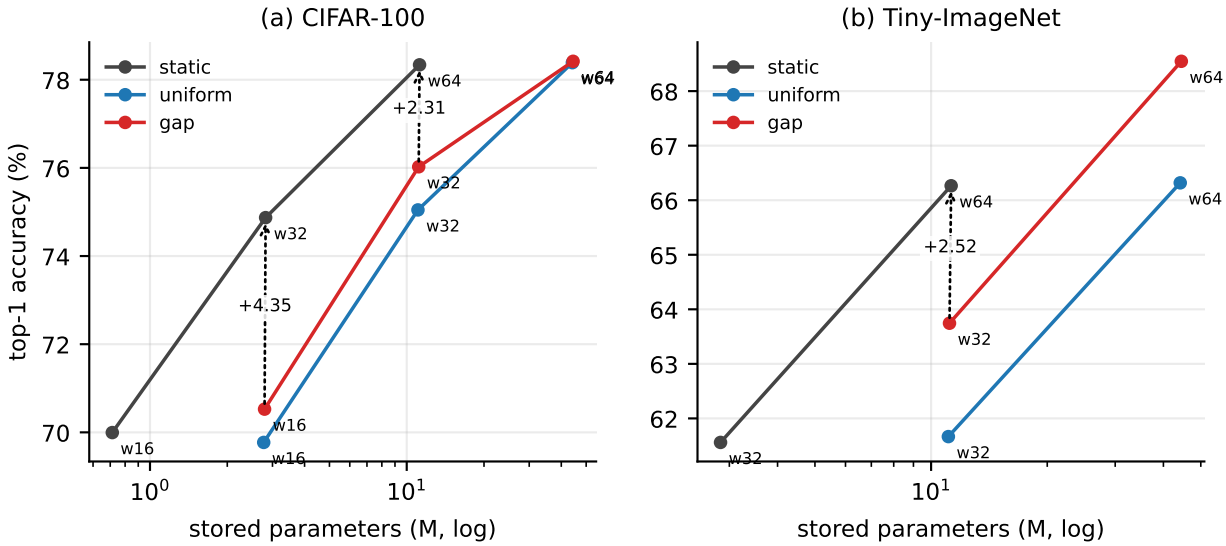


Figure 7: Accuracy against stored parameters. Dotted arrows join routed and plain networks matched to within 1 % on parameter count.

One annealing schedule. Section 11 compares a constant temperature with one linear schedule.

Stored and effective parameters. By Proposition 1 the `uniform` arm has the effective capacity of a plain network. The matched-budget comparisons of Section 10 use stored parameters, which is the count a practitioner pays for.

The CIFAR-100 width-64 null is unexplained. It is well-powered and does not depend on the temperature schedule, and its stated explanation failed a recorded test.

Provenance and corrections. Final accuracies were transcribed from the training logs, which print them verbatim, and every figure and table is generated from those per-seed values. During the analysis, three scripts grouped results by width alone, which merged runs from the two datasets and from different bank sizes; this was detected when a comparison paired one dataset's `uniform` arm with the other's `static` arm, and every number in this report was recomputed from per-seed values keyed on dataset, width, bank size and schedule. Summary sentences printed by the analysis scripts were also found to over-read non-significant orderings on four occasions; this report relies on the per-comparison statistics only. The report generator was later rewritten to read the result files directly; on the 24 width-64 result files checked it reproduces the transcribed accuracies and routing entropies exactly. The earlier version of this report printed stored parameter counts with a third decimal that had not been measured, and six of nineteen were off by up to 0.005 M (for example 0.708 M for 0.713 M); the counts here are counted from the model definition and agree with the training logs to the two decimals they print. No comparison changes.

14 Conclusion

Dynamic convolution's gains, where they appear, come from routing and not from the kernel bank: in all five conditions tested on two datasets, a bank mixed uniformly is equivalent to a plain network. Two consequences follow. Banks larger than $K = 2$ buy nothing further, and at any parameter budget we could match, the parameters are better spent on width, by 2.3 to 4.3 pp at a fraction of the training cost.

Table 6: Routed against plain networks at matched parameter counts, paired by seed. Minutes are routed / plain per run. The last column is the routed network's stored parameters divided by the plain one's; a pair is called matched only within 10 %.

dataset	routed	plain	routed	plain	Δ (pp)	95 % CI	p	min/run	param ratio
CIFAR-100	`gap` w16 (2.79M)	`static` w32 (2.82M)	70.527	74.872	+4.345	[+4.103, +4.587]	$< 10^{-4}$	33.7 / 8.6	0.99 (matched)
CIFAR-100	`gap` w32 (11.13M)	`static` w64 (11.22M)	76.025	78.338	+2.313	[+1.870, +2.757]	$< 10^{-4}$	39.2 / 18.8	0.99 (matched)
Tiny-ImageNet	`gap` w32 (11.16M)	`static` w64 (11.27M)	63.745	66.265	+2.520	[+2.311, +2.729]	$< 10^{-4}$	284.2 / 129.0	0.99 (matched)

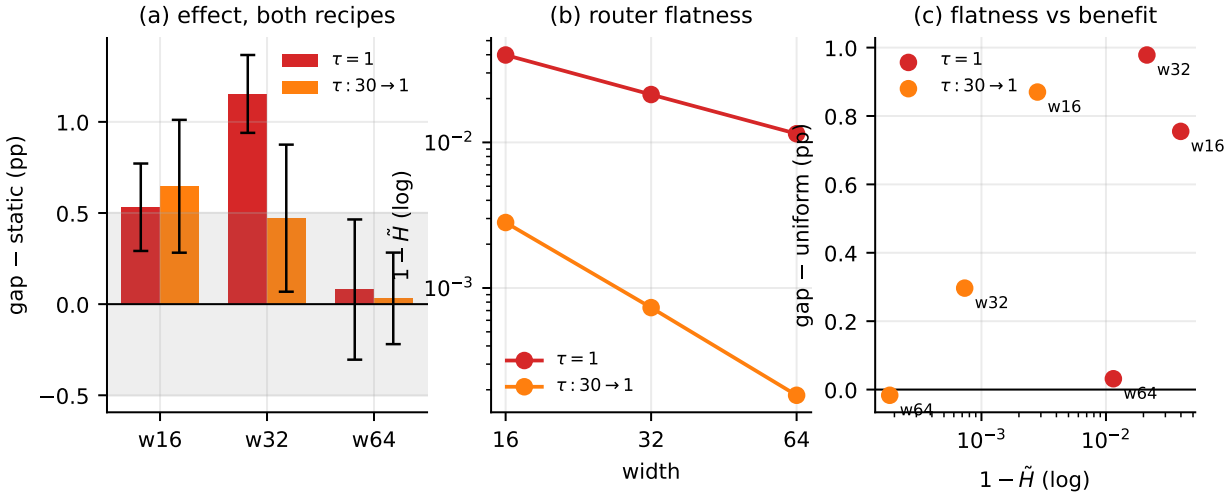


Figure 8: CIFAR-100. (a) Routing effect under a constant and an annealed temperature, 95 % CIs. (b) Router flatness. (c) Flatness against benefit over `uniform`; the two schedules do not lie on one curve.

When routing helps is less settled. On CIFAR-100 it helps below width 64 and not at 64, under either temperature schedule. On Tiny-ImageNet it helps at both widths tested. We proposed that memorisation of the training set explains the CIFAR-100 null, recorded a prediction from that account before testing it, and the prediction failed.

The diagnostic used to show that routing is active does not support that use. A router 14 times flatter by entropy delivered the larger benefit, because entropy squares the deviations that carry the routing signal.

Reproducibility. 132 runs, 204 GPU-hours, one RTX 4070 Ti. Code, the pre-registration, the dated prediction, the gate ledger with recorded evidence, and every per-seed value accompany this report.

References

S. Arora, N. Cohen, and E. Hazan. On the optimization of deep networks: Implicit acceleration by overparameterization. In *ICML*, 2018.

J. Chen, X. Wang, Z. Guo, X. Zhang, and J. Sun. Dynamic region-aware convolution. In *CVPR*, 2021.

L. Chen, L. Gu, L. Li, C. Yan, and Y. Fu. Frequency dynamic convolution for dense image prediction. In *CVPR*, pages 30178–30188, 2025.

Y. Chen, X. Dai, M. Liu, D. Chen, L. Yuan, and Z. Liu. Dynamic convolution: Attention over convolution kernels. In *CVPR*, pages 11030–11039, 2020.

X. Ding, Y. Guo, G. Ding, and J. Han. ACNet: Strengthening the kernel skeletons for powerful CNN via asymmetric convolution blocks. In *ICCV*, 2019.

X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun. RepVGG: Making VGG-style ConvNets great again. In *CVPR*, 2021.

K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

D. Lakens. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, 8(4):355–362, 2017.

Y. Le and X. Yang. Tiny ImageNet visual recognition challenge. CS231N course report, Stanford University, 2015.

Table 7: Temperature ablation on CIFAR-100, `gap` arm. The last two columns test whether the annealed router behaves like the constant mixture.

	accuracy		annealed − constant		vs `static`			annealed vs `uniform`	
width	$\tau = 1$	annealed	Δ	p	$\tau = 1$	annealed	p_{TOST}	Δ	verdict
16	70.527	70.642	+0.115	0.4313	+0.532	+0.647	0.8259	+0.870	higher
32	76.025	75.343	-0.682	0.0017	+1.153	+0.472	0.4319	+0.297	inconclusive
64	78.418	78.370	-0.048	0.7449	+0.080	+0.032	0.0025	-0.017	equivalent

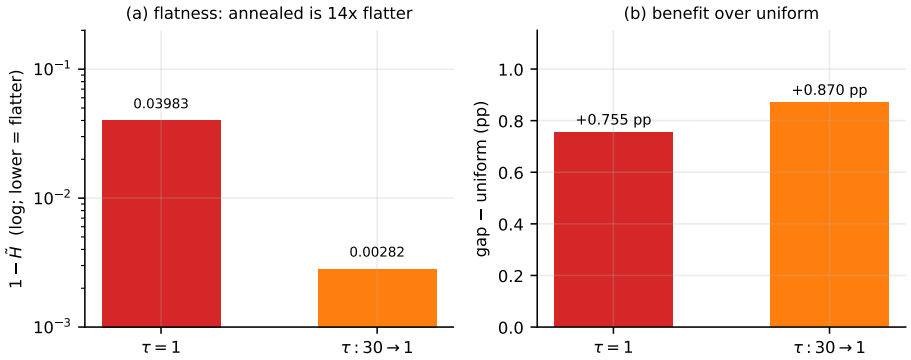


Figure 9: CIFAR-100, width 16. The annealed router is 14 times flatter by entropy and delivers the larger benefit over `uniform`.

C. Li and A. Yao. KernelWarehouse: Rethinking the design of dynamic convolution. In *ICML*, PMLR 235:29201–29221, 2024.

C. Li, A. Zhou, and A. Yao. Omni-dimensional dynamic convolution. In *ICLR*, 2022.

Y. Li, Y. Chen, X. Dai, M. Liu, D. Chen, Y. Yu, L. Yuan, Z. Liu, M. Chen, and N. Vasconcelos. Revisiting dynamic convolution via matrix decomposition. In *ICLR*, 2021.

D. J. Schuirmann. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics*, 15(6):657–680, 1987.

F. Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

B. Yang, G. Bender, Q. V. Le, and J. Ngiam. CondConv: Conditionally parameterized convolutions for efficient inference. In *NeurIPS*, 2019.

C. Zhang, S. Bengio, M. Hardt, B. Recht, and O. Vinyals. Understanding deep learning requires rethinking generalization. In *ICLR*, 2017.

Table 8: Routing entropy against the benefit it is taken to indicate, CIFAR-100. RMS dev is the root-mean-square deviation of the mixing weights from $1/K$, recovered from Eq. (4).

width	τ	$\tilde{H}$	$1-\tilde{H}$	RMS dev	`gap−uniform` (pp)	p
16	1	0.96017	0.03983	0.0831	+0.755	0.0023
16	$30\to1$	0.99718	0.00282	0.0221	+0.870	0.0044
32	1	0.97868	0.02132	0.0608	+0.978	0.0010
32	$30\to1$	0.99927	0.00073	0.0113	+0.297	0.0810
64	1	0.98853	0.01147	0.0446	+0.032	0.7752
64	$30\to1$	0.99982	0.00018	0.0056	-0.017	0.8369

A Complete results

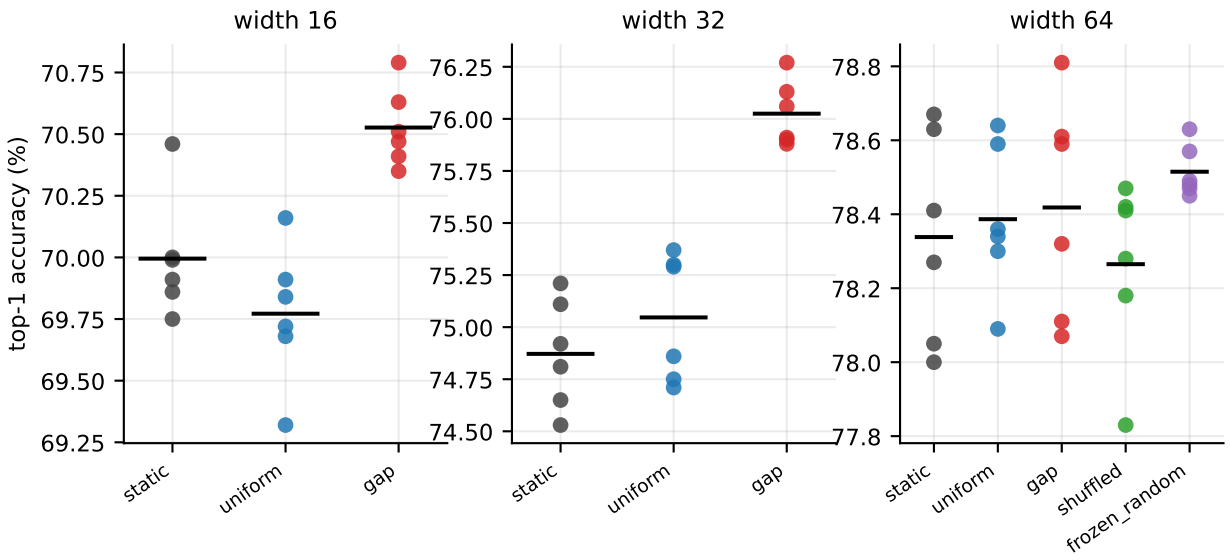


Figure 10: Every CIFAR-100 run at $K = 4$ and $\tau = 1$; bars are arm means.

Table 9: Every run at $K = 4$ and $\tau = 1$: final-epoch top-1 accuracy (%) for each seed, with mean and sample standard deviation.

dataset	width	arm	s0	s1	s2	s3	s4	s5	mean	sd
CIFAR-100	16	`static`	69.91	69.75	69.86	70.46	70.00	69.99	69.995	0.246
CIFAR-100	16	`uniform`	69.68	69.32	69.84	69.91	69.72	70.16	69.772	0.279
CIFAR-100	16	`gap`	70.35	70.51	70.41	70.63	70.79	70.47	70.527	0.160
CIFAR-100	32	`static`	74.81	74.65	74.53	74.92	75.11	75.21	74.872	0.262
CIFAR-100	32	`uniform`	75.29	74.86	74.75	75.30	74.71	75.37	75.047	0.305
CIFAR-100	32	`gap`	76.13	75.88	75.91	75.90	76.27	76.06	76.025	0.156
CIFAR-100	64	`static`	78.05	78.67	78.63	78.27	78.00	78.41	78.338	0.284
CIFAR-100	64	`uniform`	78.64	78.59	78.34	78.30	78.36	78.09	78.387	0.202
CIFAR-100	64	`gap`	78.59	78.81	78.11	78.61	78.07	78.32	78.418	0.299
CIFAR-100	64	`shuffled`	78.42	77.83	78.18	78.41	78.47	78.28	78.265	0.238
CIFAR-100	64	`frozen_random`	78.47	78.49	78.48	78.63	78.45	78.57	78.515	0.070
Tiny-ImageNet	32	`static`	61.93	61.61	61.57	61.46	61.27	61.52	61.560	0.217
Tiny-ImageNet	32	`uniform`	61.52	61.53	61.55	61.62	61.66	62.12	61.667	0.229
Tiny-ImageNet	32	`gap`	63.53	64.20	63.61	63.58	63.91	63.64	63.745	0.260
Tiny-ImageNet	64	`static`	66.29	66.59	65.88	65.97	66.63	66.23	66.265	0.309
Tiny-ImageNet	64	`uniform`	66.27	66.27	66.41	66.21	66.22	66.53	66.318	0.126
Tiny-ImageNet	64	`gap`	68.73	68.35	68.42	68.55	68.71	68.52	68.547	0.152

Table 10: Paired comparisons against `static`, $n = 6$. p_W is the Wilcoxon signed-rank test, which cannot fall below 0.031 at this n.

dataset	width	arm	Δ (pp)	95 % CI	p_t	p_W	d_z	p_{TOST}	verdict
CIFAR-100	16	`uniform`	-0.223	[-0.500, +0.054]	0.0931	0.0938	-0.85	0.0251	equivalent
CIFAR-100	16	`gap`	+0.532	[+0.292, +0.771]	0.0023	0.0312	+2.33	0.6260	higher
CIFAR-100	32	`uniform`	+0.175	[-0.146, +0.496]	0.2206	0.3125	+0.57	0.0242	equivalent
CIFAR-100	32	`gap`	+1.153	[+0.940, +1.367]	$< 10^{-4}$	0.0312	+5.67	0.9997	higher
CIFAR-100	64	`uniform`	+0.048	[-0.332, +0.429]	0.7573	0.8438	+0.13	0.0142	equivalent
CIFAR-100	64	`gap`	+0.080	[-0.305, +0.465]	0.6158	0.5625	+0.22	0.0188	equivalent
CIFAR-100	64	`shuffled`	-0.073	[-0.602, +0.456]	0.7361	1.0000	-0.15	0.0464	equivalent
CIFAR-100	64	`frozen_random`	+0.177	[-0.121, +0.474]	0.1874	0.2188	+0.62	0.0191	equivalent
Tiny-ImageNet	32	`uniform`	+0.107	[-0.270, +0.483]	0.4994	0.6875	+0.30	0.0218	equivalent
Tiny-ImageNet	32	`gap`	+2.185	[+1.781, +2.589]	$< 10^{-4}$	0.0312	+5.67	0.9999	higher
Tiny-ImageNet	64	`uniform`	+0.053	[-0.334, +0.441]	0.7380	1.0000	+0.14	0.0157	equivalent
Tiny-ImageNet	64	`gap`	+2.282	[+1.952, +2.611]	$< 10^{-4}$	0.0312	+7.26	1.0000	higher

Table 11: Configuration and cost. Router parameters are in thousands and are included in the stored total. Training loss is the final-epoch mean over six seeds.

dataset	width	arm	params (M)	router (K)	train loss	min/run
CIFAR-100	16	`static`	0.71	0.0	0.1643	9.6
CIFAR-100	16	`uniform`	2.77	0.0	0.1653	11.4
CIFAR-100	16	`gap`	2.79	21.3	0.0638	33.7
CIFAR-100	32	`static`	2.82	0.0	0.0170	8.6
CIFAR-100	32	`uniform`	11.06	0.0	0.0170	12.9
CIFAR-100	32	`gap`	11.13	74.6	0.0132	39.2
CIFAR-100	64	`static`	11.22	0.0	0.0080	18.8
CIFAR-100	64	`uniform`	44.18	0.0	0.0081	25.2
CIFAR-100	64	`gap`	44.46	288.0	0.0074	86.0
CIFAR-100	64	`shuffled`	44.46	288.0	0.0083	92.3
CIFAR-100	64	`frozen_random`	44.18	0.0	0.0077	85.4
Tiny-ImageNet	32	`static`	2.85	0.0	0.1194	54.2
Tiny-ImageNet	32	`uniform`	11.09	0.0	0.1200	56.3
Tiny-ImageNet	32	`gap`	11.16	74.6	0.0420	284.2
Tiny-ImageNet	64	`static`	11.27	0.0	0.0210	129.0
Tiny-ImageNet	64	`uniform`	44.23	0.0	0.0220	141.0
Tiny-ImageNet	64	`gap`	44.52	288.0	0.0190	687.2

Table 12: Routing entropy by arm, CIFAR-100 width 64, per-seed means.

arm	mean $\tilde{H}$	min	max	$1 - \tilde{H}$
`uniform`	1.0000	1.0000	1.0000	0.00000
`gap`	0.9885	0.9867	0.9912	0.01147
`shuffled`	0.9996	0.9995	0.9997	0.00040
`frozen_random`	0.9971	0.9967	0.9977	0.00287

Table 13: Every run in the study. GPU-hours are from logged wall-clock time.

dataset	width	K	τ	arms	runs	GPU-h
CIFAR-100	16	4	constant	`static`, `uniform`, `gap`	18	5.5
CIFAR-100	32	4	constant	`static`, `uniform`, `gap`	18	6.1
CIFAR-100	64	4	constant	`static`, `uniform`, `gap`, `shuffled`, `frozen_random`	30	30.8
CIFAR-100	32	2	constant	`gap`	6	2.8
CIFAR-100	32	8	constant	`gap`	6	6.9
CIFAR-100	16	4	annealed	`gap`	6	4.4
CIFAR-100	32	4	annealed	`gap`	6	3.8
CIFAR-100	64	4	annealed	`gap`	6	8.6
Tiny-ImageNet	32	4	constant	`static`, `uniform`, `gap`	18	39.5
Tiny-ImageNet	64	4	constant	`static`, `uniform`, `gap`	18	95.7
total					132	204.1